

tapSquares User Guide & Dictionary

Version 2 — the interpreter, its CSV grammar, and the data dictionary

The tapSquares interpreter, its CSV grammar, and the data dictionary — companion reference to the stage-by-stage Builder Guides

1. What tapSquares is

tapSquares is a NetLogo + HubNet interpreter for multiplayer, grid-based natural-resource games. A single .nlogox file acts as a stable engine; almost all game-specific behavior — actions, dynamics, scoring, information screens — is defined externally, through a fixed set of nine CSV files plus one small NetLogo file, game details.nls.

The interpreter:

- launches multiplayer HubNet sessions;
- manages player actions and effort budgets;
- handles land ownership and access logic;
- runs spatial and non-spatial update rules each round;
- calculates and displays scores;
- shows information screens between phases and rounds.

CORE DESIGN PRINCIPLE

Keep the interpreter stable. Put game logic in CSVs and in game details.nls. Only touch the .nlogox file itself when a game genuinely needs new interpreter-level machinery — which should be rare.

This guide describes the current patched template (the same one used to build the Tiger, Lantana, and Elephant/crop games shipped alongside it) and corrects and extends the earlier v1.1 draft in several places where practical experience building those three games turned up gaps or errors — most importantly, two undocumented crash conditions and a CSV-escaping rule that is easy to get wrong once and hard to diagnose afterward. Both are covered in Section 6.

2. Design philosophy

Small grammar + escape hatch

The tapSquares architecture intentionally combines a relatively small, fixed CSV grammar with designated expression fields where custom NetLogo code can be dropped in. This avoids endlessly expanding the interpreter's own grammar while still letting each game define whatever logic it actually needs.

Expression-bearing fields include trigger, expression, property, flags, parameter, onlyIf, and notIf. These columns contain executable NetLogo code fragments — not configuration values in the ordinary sense.

IMPORTANT FRAMING

tapSquares CSV files are not just configuration. Together, they form a lightweight rule language, and the expression fields are where that language's real logic lives.

Where long or reused logic belongs

There are three places to put logic that's too long, or too often repeated, to comfortably live inline in a single CSV cell:

- `spatialFunctionList.csv` and `nonSpatialFunctionList.csv`, for one conceptual rule per row, in an ordered sequence — the intended home for most game dynamics;
- a to-report reporter procedure declared in `game details.nls`, referenced by its bare name from any expression field that expects a value. Since a zero-argument to-report is itself a valid NetLogo expression, any CSV field that runs as an expression can simply name the reporter instead of repeating a long formula. See Section 8.8 for a worked example — the same expression used in both a scoring rule and an information screen, declared once.
- occasionally, a to command procedure in `game details.nls`, for a multi-step action too long to read comfortably as one CSV cell — referenced the same way, by name, from an expression field.

Recommended conventions when writing rows in the two function-list files: one conceptual rule per row; keep rules in a deliberate order (see Section 9 on reset/accumulate/act ordering); avoid magic numbers — pull them from `sessionList.csv` instead (Section 8.1); and if a row's condition can never actually fire given the rest of the file, leave it in place with a comment explaining why, rather than deleting it (Section 9).

3. game details.nls — declarations and the three hooks

Every `tapSquares` game ships its own `game details.nls` alongside the nine CSV files. This is the one file that routinely changes from game to game, and — outside of genuinely new interpreter features — should be the only NetLogo file you need to write.

Variable declarations

New games will usually need new globals, patches-own, or turtles-own variables. These are declared at the top of `game details.nls` exactly as they would be in the main model:

```
globals [  
  pDegraded_increment  
]  
  
patches-own [  
  hasResource  
  degraded  
  totalHarvestedLight  
  totalHarvestedHeavy  
  totalCropped  
  pDegraded  
]
```

(This example, and the rest of this section, is Lantana's actual `game details.nls`, lightly trimmed.)

The three hooks, and when the engine calls them

game details.nls defines three procedures that the engine's own start-game calls at fixed points. Knowing exactly where each one sits in the startup sequence matters, because it determines what state is and isn't available yet:

Hook	Called...	Context
set-initial-player-conditions	right after the engine sets its own generic per-player defaults (playerConfirm, playerCurrentScore, playerCurrentResources, playerTotalScore)	observer context
set-initial-patch-conditions	inside the engine's own per-patch setup loop, right after generic patch defaults (playerAccess, currentChoice, ownedBy)	patch context — runs once per in-game patch
set-additional-game-level-conditions	after village/private land setup, and before the engine's own set-initial-conditions	observer context

ORDERING CONSEQUENCE

Because set-additional-game-level-conditions runs before the engine's own CSV-driven set-initial-conditions (the initialConditionList.csv processor described in Section 8.3), any state you set in these three hooks is visible to that CSV file's onlyIf/notIf checks. The reverse isn't true: the hooks themselves run too early to see whatever set-initial-conditions is about to randomly assign.

Full worked example — Lantana's game details.nls:

```
to set-initial-player-conditions
  ;; The old template's "GAME SPECIFIC PLAYER LEVEL VARIABLES" block in
  ;; start-game was empty -- Lantana has no custom per-player variables.
end

to set-initial-patch-conditions
  set hasResource false
  set degraded false
  set totalHarvestedLight 0
  set totalHarvestedHeavy 0
  set totalCropped 0
  set pDegraded 0
end

to set-additional-game-level-conditions
  ;; Village/private land setup is handled generically by the engine;
  ;; nothing further is required here for Lantana.
end
```

A new game's routine build sequence is therefore: copy a working folder, add the variable declarations, write the three hooks (even if some are empty, as above), then move on to the CSVs.

4. How CSV files are read

General pattern for every *List.csv file: the first row is column headers; each subsequent row creates one NetLogo agent of the relevant type (an action, a spatial function, a scoring rule, and so on); headers map directly to that agent's variable names.

NetLogo's csv extension auto-types each cell as it's read — numbers become numbers, TRUE/FALSE become booleans, and anything else stays a string. What happens next is the part worth understanding precisely, because it's the root of the two rules in Section 6. For each cell that comes through as a string, the engine builds and runs a small piece of code to assign it:

```
ifelse is-string? ?2 [  
  set currentParameter (word "set " ?1 " \" ?2 "\"")  
][  
  set currentParameter (word "set " ?1 " " ?2)  
]  
run currentParameter
```

In other words: a string-valued cell doesn't just get assigned — it gets wrapped in an outer pair of quotes and then run as NetLogo code. This is exactly why expression-bearing fields (trigger, expression, flags, and the rest) must themselves be quoted strings inside the CSV: the wrapping only happens for cells NetLogo already reads as strings, and only a quoted string field survives as one all the way through the CSV parser.

5. Main CSV files

File	Purpose
sessionList.csv	game/session parameters and file references
actionList.csv	player actions
initialConditionList.csv	starting landscape
stateList.csv	patch states and their visuals
turtleList.csv	countable display objects
spatialFunctionList.csv	patch-level and neighbor dynamics
nonSpatialFunctionList.csv	global / multi-agent dynamics
scoringFunction.csv	scoring
infoScreenList.csv	information screens

6. Two rules that will crash your game if you miss them

Everything in this section was learned the hard way, building Tiger and Lantana this year, and neither rule was documented in the earlier v1.1 draft. Both are worth reading closely — they account for essentially every load-time or round-advance crash encountered in practice.

6.1 `actionList.csv` and `infoScreenList.csv` need contiguous IDs

Every other `*List.csv` file — `spatialFunctionList`, `nonSpatialFunctionList`, `scoringFunction`, `stateList`, `turtleList`, `initialConditionList` — is read by sorting on its ID column and iterating over whatever rows exist. Gaps are harmless in these files: `functionID` 1, 2, 5, 9 works fine.

`actionList.csv` and `infoScreenList.csv` are different. Both are read using their ID column as a dense counter rather than a sorted list. Here is the actual code, from `read-action-file`:

```
set actionEffortList n-values (count actions) [0]
foreach n-values (count actions) [ ?1 -> ?1 + 1 ] [ ?1 ->
  ask one-of actions with [actionID = ?1] [
    set actionEffortList replace-item (?1 - 1) actionEffortList effort
  ]
]
```

and the equivalent from `advance-to-next-round / show-information-screen`:

```
set currentInformationScreen currentInformationScreen + 1
...
show-information-screen currentInformationScreen
;; inside show-information-screen:
let currentScreen one-of infoScreenFunctions with [screenID = currentScreenID]
```

If a game has three actions numbered 1, 2, and 4 (skipping 3), `count(actions)` is 3, so the loop checks IDs 1, 2, and 3 — action 3 doesn't exist, `one-of actions with [actionID = 3]` returns `NOBODY`, and `ask NOBODY` crashes the game immediately on load. The same failure mode hits `infoScreenList.csv`'s `screenID`, except it surfaces the first time the game tries to advance past a round rather than on load.

RULE

Keep `actionID` and `screenID` contiguous, starting at 1, with no gaps — in every version of these two files you ship, including any intermediate build stages if you're staging a game for teaching purposes.

6.2 Nested quotes inside expression fields

Recall from Section 4 that any string-valued cell gets wrapped as `set <field> "<value>"` before being run. If `<value>` itself needs to contain a literal quoted string — for instance, an expression that sets a shape, `set shape "lion"` — the wrapping would produce `set X "set shape "lion""`, which is broken: the quotes close early and the rest is left dangling.

The fix is to escape the inner quotes in the CSV so they survive the wrap as literal backslash-quotes. A `spatialFunctionList.csv` expression cell that needs to sprout a turtle shaped like a lion should be authored, inside the CSV's own quoting, as:

```
"sprout-displayTurtles 1 [set shape \"\"lion\"\" set size 1 set mobile? true]"
```

The general rule: wherever the NetLogo code inside an expression field needs a literal "word", write `\"\"word\"\"` in the CSV cell. This class of bug is easy to introduce once and hard to diagnose afterward — the symptom is usually a shape or property silently failing to apply, not a hard crash — so the safest approach is to copy a known-working row's quoting pattern rather than writing a new one from scratch.

7. Three kinds of CSV values

Simple parameters

Examples: `numRounds = 3`, `effort = 5`, `endowment = 20`. Usually safe to edit directly.

Lists

Examples: `round_play = 1 2 3 4`, `phaseAvailable = 1 2`. Parsed into NetLogo lists automatically.

Executable expressions

Examples: `trigger = degraded`, `expression = set pDegraded (pDegraded + pDegraded_increment)`. These are executable NetLogo fragments, and must be quoted strings in the CSV (see Sections 4 and 6.2).

Literal values generally should not be quoted (0.25, TRUE, uniform, integer); expression-field values generally should be (`"hasGrass"`, `"soilQuality > 0.7"`).

8. Data dictionary

8.1 sessionList.csv

Purpose: game/session setup, central parameter bank, and file references. Baseline columns:

Column	Type	Description
sessionID	number/text	Session identifier.
gameID	number/text	Game identifier.
numRounds	number	Number of rounds.
numPhases	number	Number of phases.
chooseVillageLand?	boolean	Whether village/shared-land selection occurs.
choosePrivateLand?	boolean	Whether private-land selection occurs.
villageMembership	list	Player-group assignments.
maxPrivateSquares	number	Maximum private patches.

Column	Type	Description
endowment	number	Player effort budget.
actionListFileName / initialConditionListFileName / stateListFileName / spatialFunctionListFileName / scoringFunctionFileName / infoScreenListFileName / turtleListFileName / nonSpatialFunctionListFileName	string	File references for the other eight CSVs.
patch_color_min / patch_color_max	RGB list	Lower/upper patch-color bounds.
patch_color_property	expression	Patch-color scaling expression. Safe to leave blank when min and max are equal — there's nothing to interpolate.

Beyond this baseline, `sessionList.csv` doubles as an open-ended parameter bank: any game-specific global gets added as an ordinary extra column, in whatever order is convenient, alongside the fixed ones. Tiger's actual header shows this clearly — `numAnimals`, `grassK`, `grassR`, `minGrassMass`, `pAttack_increment`, `grassPerTurnEat`, `pDie_unfed`, `cost_offFarmIncome`, `mult_offFarmIncome`, and `grassMass_forage` all sit interspersed among the baseline columns, not appended in a separate block. Treat `sessionList.csv` as the single place any tunable, game-wide number lives — anything a facilitator might want to change between sessions without touching a spatial or scoring rule.

8.2 actionList.csv

Column	Type	Description
actionID	number	Unique action identifier. Must be contiguous starting at 1 — see Section 6.1.
imageName	string	Action image/shape.
round_play	list	Active rounds.
phaseAvailable	list	Active phases.
effort	number	Resource cost.
flags	expression list	Conditions for action availability — may hold several conditions, e.g. "hasResource" and "not degraded", the same way onlyIf/notIf work in Section 8.3.

Actions can fail to appear for any of several reasons: wrong round, wrong phase, insufficient effort remaining, no land access, or a flags condition that isn't met. Check all five before assuming something is broken.

8.3 initialConditionList.csv

Purpose: define initial landscape conditions, separating what variable is initialized, where initialization is allowed, and how values are generated.

Column	Type	Description
conditionID	number	Condition identifier/order.
variable	variable name	Patch variable to initialize.
conditionType	string	Conceptual variable type (boolean, numeric, etc.). This column is read into a variable but is not actually branched on anywhere in the current engine — it is documentation only today, not validation, despite appearances.
probability	number	Probability of assignment among eligible patches.
onlyIf	list of reporter expressions	All must evaluate TRUE for initialization to occur.
notIf	list of reporter expressions	All must evaluate FALSE for initialization to occur.
valueMode	string	fixed, expression, uniform, or integer.
valueExpression	expression/string	Expression or literal value used for initialization.
minValue / maxValue	number/expression	Bounds for the uniform and integer modes.

Conceptually, each row works as: if probability succeeds, and all onlyIf conditions are TRUE, and all notIf conditions are FALSE, then initialize variable according to valueMode. Value modes: fixed assigns valueExpression directly; expression evaluates valueExpression with runresult; uniform draws a random float between minValue and maxValue; integer draws a random integer in the same range.

Example rows:

```
conditionID,variable,conditionType,probability,onlyIf,notIf,valueMode,valueExpression,minValue,maxValue
1,hasGrass,boolean,0.25,,,fixed,TRUE,,
2,hasCanola,boolean,1,,"hasGrass",fixed,TRUE,,
3,soilQuality,numeric,1,,,uniform,,0.7,1.0
4,grassCount,numeric,1,"hasGrass",expression,"soilQuality * fullGrass",,
```

8.4 stateList.csv

Column	Type	Description
stateID	number	State identifier/order.
flags	expression list	Conditions defining the state.

Column	Type	Description
imageName	string	Shape/image.
stateColor	color	Optional display color for the state.

8.5 turtleList.csv

Purpose: define countable display objects. The current column set (corrected from v1.1, which listed a single turtleSize):

Column	Type	Description
turtleID	number	Display-object identifier.
countMeasure	expression	Quantity to display.
imageName	string	Shape/image.
mobile?	boolean	Whether the object moves.
transparent?	boolean	Optional. Used by the engine's own information-screen display logic to dim or hide certain display turtles during summary screens. If the column is omitted, the underlying variable still exists and defaults to false.
turtleSizeMin / turtleSizeMax	number	Display size range (replaces a single turtleSize in older games).

8.6 spatialFunctionList.csv

Column	Type	Description
functionID	number	Function order/identity.
trigger	expression	Trigger condition, evaluated on each in-game patch.
localVariables / localTargets	list	Local variable bindings, if needed.
expression	command expression	Rule executed when triggered.
round_play / phase	list	Active rounds / phases.
xOffset / yOffset	number	Neighbor offset from the triggering patch.

Verified conceptual model: for each in-game patch (the source), if trigger evaluates true there, expression is run in the context of patch-at xOffset yOffset (the target). An offset of (0, 0) means source and target are the same patch. The engine checks that the target patch actually exists before running the expression, so an offset that would fall off the edge of the grid is silently skipped rather than causing an error — worth knowing if you're deliberately building something at the world's boundary.

Worked example (Lantana) — four rows, one per compass direction, each triggered by degradation on the source patch and pushing degradation risk onto one neighbor:

```
functionID,trigger,...,expression,...,xOffset,yOffset
2,"degraded","...","set pDegraded (pDegraded + pDegraded_increment)","...",-1,0
3,"degraded","...","set pDegraded (pDegraded + pDegraded_increment)","...",1,0
4,"degraded","...","set pDegraded (pDegraded + pDegraded_increment)","...",0,-1
5,"degraded","...","set pDegraded (pDegraded + pDegraded_increment)","...",0,1
```

8.7 nonSpatialFunctionList.csv

Column	Type	Description
functionID	number	Function order/identity.
expression	command expression	Global rule block.
round_play / phase	list	Active rounds / phases.

Typical uses: moving animals or players, redistributing resources, player-level bookkeeping, and other logic that doesn't belong to any single patch.

8.8 scoringFunction.csv

Column	Type	Description
scoreID	number	Score-rule order/identity.
patchLevel?	boolean	Whether scoring occurs at the patch level.
trigger	expression	Trigger condition.
property	expression	Quantity to score — see note below.
value	number	Multiplier or assigned value.
localTarget	variable	Patch-level target.
playerTarget	variable/list	Player-level target.
loop?	boolean	Whether to loop through players.
playerMarker	expression	Player assignment logic.
operation	string	Score operation (e.g. set, add).
show? / imageName / column	boolean/string/number	Score-display settings.
round_play / phase	list	Active rounds / phases.

PROPERTY CAN NAME A REPORTER

Since a zero-argument to-report procedure is itself a valid NetLogo expression, property can simply be

that reporter's bare name instead of repeating a formula. Lantana declares total-patch-score-components once in game details.nls and references it from both scoringFunction.csv (scoreID 7) and infoScreenList.csv (screenID 1) — one formula, defined in one place, used in two CSVs:

```
scoreID,patchLevel?,trigger,property,value,...,playerMarker,operation,show?,...  
7,TRUE,"selected?","total-patch-score-components",1,...,selectedBy,add,TRUE,...
```

This file currently combines score generation, transfer, assignment, and display in one place. That's flexible, but it's worth deliberately ordering rows — resets first, accumulation next, display-affecting rows last — rather than relying on the file staying simple. See Section 9.

8.9 infoScreenList.csv

Column	Type	Description
screenID	number	Screen identity/order. Must be contiguous starting at 1 — see Section 6.1.
level	string	Display level. In practice, always one of patch or player across every current tapSquares game.
parameter	expression	Value to display — may be a bare reporter name, exactly as in Section 8.8.
round_play / phase	list	Active rounds / phases.

Information screens can be made conditional on round and phase, which enables tutorial-only rounds, delayed feedback, and experimental information treatments where different sessions see different summary information.

PLAYER-LEVEL SCREENS DOUBLE AS CLEANUP

A player-level info screen clears every mobile displayTurtle-here before drawing its summary. If a game sprouts a transient, display-only sprite (an attack indicator, for instance) that isn't tracked in turtleList.csv, the only way it gets cleared is by this mechanism — and it only runs once a player-level info screen is active in that same round. A spatialFunctionList.csv row aimed at killing the sprite won't help, because spatial functions run once at the very start of the next round, leaving the sprite visible for the entire round it appeared in. If you need a transient sprite to disappear within the round it's drawn, make sure a player-level info screen is already active in that round.

9. Common patterns and conventions

Reset, accumulate, act

Within `spatialFunctionList.csv` and `scoringFunction.csv`, rows execute in ID order. Give reset rows (zeroing a per-round counter) the lowest IDs, accumulation rows the next, and action or display rows the highest — this keeps the file readable as a sequence, and avoids the classic bug of a reset row accidentally running after the value it's supposed to be resetting has already been used.

Scattering after move-to

`move-to` snaps a turtle to the exact center of its destination patch. If animals or players should look randomly distributed within a patch rather than stacked at its center, follow `move-to` with a scatter:

```
move-to target-patch  
setxy (pxcor - 0.5 + random-float 1) (pycor - 0.5 + random-float 1)
```

Document inert code instead of deleting it

If a row's condition can, on inspection, never actually fire given the rest of the file — Lantana has exactly one such row — leave it in place with a comment explaining the reasoning, rather than silently removing it. A later edit to an earlier row could reactivate the condition, and the comment preserves why it was left alone rather than forcing the next person to re-derive the same reasoning from scratch.

Look-ahead declarations

When staging a game into teaching build-stages, it's often cleaner to declare a `patches-own` variable (with a default that preserves current behavior) a stage or two before it becomes functionally meaningful, rather than deferring the declaration and having to reorder or rewrite an earlier stage's logic later. Tiger's `fenced?` variable is declared this way, several stages before fencing mechanics actually use it.

10. Safe vs advanced editing

Usually safe

- rounds/phases; effort values; probabilities; image names; scoring multipliers.

Intermediate

- IDs (mind Section 6.1); lists; display settings.

Advanced

- expressions; triggers; scoring targets; custom logic in `game details.nls`.

11. Builder workflow

1. Copy a working game folder.
2. Define new variables and write the three hooks in `game details.nls` (Section 3), even if some are empty.

- 3. Configure sessionList.csv.
- 4. Define actions (mind contiguous actionIDs, Section 6.1).
- 5. Define initial conditions.
- 6. Add spatial dynamics.
- 7. Add non-spatial dynamics.
- 8. Add scoring.
- 9. Add information screens (mind contiguous screenIDs, Section 6.1).
- 10. Test incrementally.

12. Testing workflow

- 1. Session loads without error.
- 2. actionID and screenID are contiguous starting at 1 (Section 6.1) — check this before anything else if load or round-advance crashes.
- 3. Landscape initializes as expected.
- 4. Actions appear correctly, in the right rounds and phases.
- 5. Actions cost the correct effort.
- 6. Actions produce their intended effects.
- 7. Spatial dynamics update correctly, including at the edges of the grid.
- 8. Scoring updates correctly across a full round.
- 9. Information screens display correctly, and transient sprites clear when expected (Section 8.9).
- 10. Multiplayer access and land ownership work as intended.
- 11. The full game runs start to finish without manual intervention.

13. Troubleshooting

ASK expected input to be an agent or agentset but got NOBODY — in READ-ACTION-FILE

actionList.csv's actionID column has a gap. Renumber it contiguously starting at 1. See Section 6.1.

OF expected input to be a turtle agentset or turtle but got NOBODY — in SHOW-INFORMATION-SCREEN

infoScreenList.csv's screenID column has a gap. Renumber it contiguously starting at 1. See Section 6.1.

Session does not load

- Check file names, sessionID, CSV headers, and for any missing files.

Action does not appear

- Check round, phase, effort, land access, and flags.

Action appears but does nothing

- Check spatial/non-spatial functions, currentChoice references, trigger conditions, and function order.

A display sprite doesn't clear, or clears a round late

- See the player-level-screens-double-as-cleanup note in Section 8.9 — a spatialFunctionList row is almost never the right fix for this.

Scores are wrong

- Check scoring order, reset rows, triggers, and player assignment logic.

A shape or property silently doesn't apply

- Check for the nested-quote escaping issue in Section 6.2 before assuming the trigger or condition is wrong.

Other NetLogo errors

- Most common causes: undeclared variables, malformed lists, invalid expressions, missing image names, mismatched IDs.

14. Worked examples in this release

This guide ships alongside three complete tapSquares games, each with its own stage-by-stage Builder Guide walking through its construction from an empty template. This User Guide is the reference companion to those three guides — use it to look up what a column or pattern means; use the Builder Guides to see a full game assembled step by step.

Game	What it demonstrates
Elephant/crop game	The original tapSquares game and the one the first Builder Guide was written against — a human-wildlife crop-conflict game and the baseline reference for the engine's core patterns.
Tiger	Fence-then-attack-roll functionID ordering; the reset/accumulate pattern for a per-round loss counter; the player-level-info-screen cleanup mechanism for a transient display sprite; look-ahead variable declarations across build stages.
Lantana	The reporter-as-property consolidation technique; documenting inert code rather than deleting it; the neighbor-offset "push a condition onto a neighbor" spatial-function pattern; a full worked example of porting an older CSV schema onto the current template.

15. Recommended future improvements

- A minimal template game — an empty, already-wired-up starting point for a brand-new game.

- A variables.md template, so each game documents its own variables' scope, type, meaning, and where each is used.
- An expression cookbook of worked examples for the trickier fields (quoting, escaping, reporter references).
- A small CSV validation script — even just a contiguity check on actionID and screenID before a session loads would directly prevent the single most common crash found in practice this year.
- Optional documentation-only columns (label, description, notes) across the *List.csv files, to make each row's intent legible to a collaborator without having to reverse-engineer it from the expression alone.